ripopt: An Interior Point Optimizer in Rust with Implicit Slacks and Robust Recovery

John R. Kitchin

March 30, 2026

Note: This manuscript is generated and updated by Claude Code, based on the ripopt codebase and its development history. The content reflects the design decisions, algorithmic innovations, and benchmarking results that emerged during the implementation process. The manuscript is structured to provide a comprehensive overview of ripopt's architecture, key contributions, and performance relative to Ipopt. The author (John R. Kitchin) provided high-level guidance and review but did not write any of the code or manuscript text directly (besides this note).

See supporting-information.org for links to the full source code, test suite, and benchmarking infrastructure.

Contents

1 Abstract **4**

2 Introduction **4**
2.1 Nonlinear Programming and Interior Point Methods 4
2.2 The Ipopt Algorithm 5
2.3 Motivation for ripopt 5
2.4 Contributions 6
2.5 Notation and Conventions 7

3 Implicit Slack Formulation **7**
3.1 Standard Approach: Explicit Slacks 7
3.2 Our Approach: Implicit Slacks 8
3.3 System Size Reduction 8
3.4 Handling Infeasible Iterates 9
3.5 Condensed KKT via Schur Complement 9

4 Convergence with Stationarity-Based Bound Multipliers **9**
4.1 Challenges Without Explicit Slacks 9
4.2 Stationarity-Based Optimal Multipliers 10
4.3 The Complementarity Gate 10
4.4 Two-Layer Convergence Test 10

5 Two-Phase Restoration with Multi-Attempt Recovery **11**
5.1 Background . . . 11
5.2 Phase 1: Gauss-Newton Feasibility Solver . . . 11
5.3 Phase 2: NLP Restoration . . . 12
5.4 Multi-Attempt Recovery . . . 12
5.5 Post-Restoration State Reset . . . 13

6 Automatic NE-to-LS Reformulation **13**
6.1 Problem Setting . . . 13
6.2 Detection . . . 13
6.3 Reformulation as Least Squares . . . 14
6.4 Full Hessian with Second-Order Correction . . . 14
6.5 Fallback and Result Interpretation . . . 14

7 Preprocessing **15**
7.1 Fixed Variable Elimination . . . 15
7.2 Redundant Constraint Detection . . . 15
7.3 Bound Tightening from Linear Constraints . . . 15
7.4 Preprocessing Fallback . . . 16

8 Near-Linear Constraint Detection **16**

9 Multi-Solver Fallback Architecture **16**
9.1 L-BFGS for Unconstrained Problems . . . 16
9.2 Augmented Lagrangian for Constrained Problems . . . 17
9.3 SQP Fallback for Constrained Problems . . . 17
9.4 Explicit Slack Reformulation . . . 17
9.5 Best-Point Tracking for Cycling Detection . . . 18

10 Adaptive Barrier Parameter **18**
10.1 Simplified Oracle . . . 18
10.2 Free and Fixed Modes . . . 18
10.3 Additional Design Choices . . . 19

11 Filter Line Search **19**
11.1 Switching Condition . . . 19
11.2 Second-Order Correction . . . 19
11.3 Objective Safeguard . . . 20

12 Linear Algebra **20**
12.1 Bunch-Kaufman Factorization . . . 20
12.2 Sparse LDL^T Factorization . . . 20
12.3 Inertia Correction . . . 20
12.4 Iterative Refinement . . . 21
12.5 KKT-Aware Factorization for Indefinite Systems . . . 21
12.6 Nested Dissection Ordering . . . 22

12.7 Iterative Solver: Preconditioned MINRES . . . 23
12.8 Hybrid Direct-Iterative Solver . . . 23
12.9 Configurable Stall Detection . . . 24

13 Implementation **24**
13.1 Language and Dependencies . . . 24
13.2 Problem Interface . . . 24
13.3 Performance Characteristics . . . 24
13.4 AMPL NL Interface and Pyomo Integration . . . 25
13.5 C API . . . 25
13.5.1 Design . . . 25
13.5.2 Usage Example . . . 26
13.5.3 Memory Safety and Thread Safety . . . 26
13.6 GAMS Solver Link . . . 26
13.7 Julia/JuMP Interface . . . 27

14 Computational Results **27**
14.1 Hock-Schittkowski Test Suite . . . 28
14.1.1 Solve Rates . . . 28
14.1.2 Solution Quality . . . 28
14.1.3 Speed . . . 28
14.1.4 Iteration Counts . . . 28
14.2 CUTEst Benchmark Suite . . . 28
14.2.1 Overall Solve Rates . . . 29
14.2.2 Solution Quality . . . 29
14.2.3 Speed Comparison . . . 29
14.2.4 Attribution of ripopt-only Solves . . . 30
14.3 Domain-Specific Benchmarks . . . 30
14.3.1 Electrolyte Speciation . . . 30
14.3.2 AC Optimal Power Flow . . . 31
14.3.3 Parameter Estimation . . . 31
14.4 Failure Analysis . . . 31
14.4.1 Common Failure Patterns . . . 31
14.4.2 Comparison of Failure Modes . . . 31
14.5 Speed vs. Accuracy Trade-off . . . 32

15 Parametric Sensitivity Analysis **32**
15.1 Implementation . . . 33
15.2 Example: HS071 with Parametric Constraint Bound . . . 33

16 Opportunities for Improvement **33**
16.1 Infeasibility Detection and Recovery . . . 34
16.2 Adaptive Barrier Parameter Tuning . . . 34
16.3 Multiplier Initialization . . . 34
16.4 Performance Optimization for Large Problems . . . 34

16.5 Quasi-Newton as Primary Solver 35
16.6 Scaling 35
16.7 Summary of Estimated Impact 35

17 Related Work **36**

18 Conclusions **36**

19 References **37**

1 Abstract

We present ripopt, a nonlinear interior point optimizer implemented in Rust. ripopt originated as a translation of the Ipopt algorithm but has diverged in several architectural and algorithmic respects. The core design decision—eliminating explicit slack variables for inequality constraints in favor of an implicit slack formulation—produces smaller Karush-Kuhn-Tucker (KKT) systems and a simplified problem interface. This formulation necessitates novel strategies for convergence detection, including stationarity-based bound multipliers with a complementarity gate, and a two-layer convergence test. We introduce a two-phase restoration architecture that uses a fast Gauss-Newton feasibility solver before falling back to a full NLP restoration subproblem, combined with a multi-attempt recovery strategy that explores different barrier landscapes through systematic perturbation. For nonlinear equation systems posed as constrained optimization, ripopt automatically detects the problem structure and reformulates it as an unconstrained least-squares problem with a full Hessian including second-order corrections. A multi-solver fallback architecture—including L-BFGS for unconstrained problems, augmented Lagrangian for constrained problems, and explicit slack reformulation—provides additional robustness when the primary IPM fails. ripopt also provides sIPOPT-style parametric sensitivity analysis, computing how the optimal solution changes under parameter perturbations via a single KKT backsolve at near-zero cost. On the Hock-Schittkowski test suite, ripopt solves 118 of 120 problems (98.3%) compared to Ipopt's 116 (96.7%). On the CUTEst benchmark suite of 727 problems, ripopt solves 569 (78.3%) compared to Ipopt's 556 (76.5%), with a geometric mean wall-clock speedup of 10.2× on commonly-solved problems. On domain-specific benchmarks, ripopt solves all 13 electrolyte speciation problems (vs. 12/13 for Ipopt) and matches Ipopt on AC optimal power flow (4/4). We provide a detailed analysis of where each solver excels, the failure modes that remain, and opportunities for improvement.

2 Introduction

2.1 Nonlinear Programming and Interior Point Methods

We consider the general nonlinear programming (NLP) problem:

$$
\begin{aligned}
\min_{x \in \mathbb{R}^n} \quad & f(x) \\
\text{s.t.} \quad & g_L \leq g(x) \leq g_U \\
& x_L \leq x \leq x_U
\end{aligned}
\tag{1}
$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is the objective, $g : \mathbb{R}^n \to \mathbb{R}^m$ are the constraints, and x_L, x_U, g_L, g_U are bounds (with $\pm\infty$ permitted). Interior point methods[1,2] replace the bound constraints with logarithmic barrier terms and solve a sequence of barrier subproblems parameterized by $\mu > 0$:

$$
\min_x \; f(x) - \mu \sum_i \ln(x_i - x_{L,i}) - \mu \sum_i \ln(x_{U,i} - x_i) \quad \text{s.t.} \quad g_L \leq g(x) \leq g_U \tag{2}
$$

As $\mu \to 0$, the barrier subproblem solutions converge to a solution of the original NLP. The first-order optimality conditions of the barrier problem yield the primal-dual system, which is solved by Newton's method at each iteration.

2.2 The Ipopt Algorithm

Ipopt (Interior Point OPTimizer)[3] is the most widely used open-source solver for large-scale nonlinear programming. It combines a primal-dual interior point method with a filter line search[4] and an adaptive barrier parameter update strategy.[5] Key design decisions in Ipopt include: (i) introduction of explicit slack variables for every inequality constraint, (ii) a quality-function-based barrier parameter oracle with golden-section search, (iii) a single NLP restoration phase for feasibility recovery, and (iv) reliance on external sparse symmetric indefinite linear solvers (MA27, MA57, MUMPS, or Pardiso).

Ipopt has proven highly effective across a wide range of applications, from chemical process optimization to robotics trajectory planning. However, its design carries computational overhead from explicit slack variables, complex barrier parameter selection, and the initialization and interop costs of Fortran-based linear algebra libraries. These overheads are particularly significant for small-to-medium problems where the per-iteration cost is dominated by setup rather than factorization.

2.3 Motivation for ripopt

This project began as an attempt to translate the Ipopt algorithm into Rust, motivated by these goals:

1. **Explore the limits of agentic code engineering** Claude Code was used exclusively to generate all the code in ripopt, including the core IPM engine, linear algebra routines, and the entire test suite and benchmarking infrastructure. This project serves as a case study in how far we can push agentic code generation for complex numerical algorithms.

2. **Eliminate external dependencies**. Ipopt requires Fortran linear algebra libraries (MUMPS, MA27/57, or Pardiso) that complicate installation, cross-compilation, and embedding in modern software systems. A pure Rust implementation with built-in linear solvers removes this friction.

3. **Reduce per-iteration overhead**. Rust's zero-cost abstractions, stack allocation, and absence of garbage collection enable lower per-iteration times, especially for small problems where C/Fortran interop and dynamic memory allocation dominate.

We almost immediately learned a direct, line by line translation did not occur through agentic engineering. Instead, the development process was highly iterative and exploratory, with many design decisions revisited multiple times as we gained a deeper understanding of the algorithm and its failure modes. This led to several significant divergences from the original Ipopt design, introducing several algorithmic innovations that ultimately allowed it to solve more benchmark problems than the reference implementation. These were not all intentional—some were serendipitous discoveries that emerged from the process of debugging and improving the implementation—but they represent meaningful contributions to the state of the art in interior point optimization. Ultimately, we kept these innovations because they improved robustness and performance, even though they made the code less of a direct translation of Ipopt.

2.4 Contributions

The contributions of this paper are:

1. An implicit slack formulation for inequality constraints that reduces the KKT system dimension from $(n + m + n_{\text{ineq}})$ to $(n + m)$ and simplifies the algorithmic interface (Section Implicit Slack Formulation).

2. A stationarity-based bound multiplier computation with a complementarity gate for robust convergence detection without explicit slack variables (Section Convergence with Stationarity-Based Bound Multipliers).

3. A two-phase restoration architecture combining a fast Gauss-Newton feasibility solver with a full NLP restoration subproblem, and a multi-attempt recovery strategy that systematically perturbs the barrier landscape (Section Two-Phase Restoration with Multi-Attempt Recovery).

4. Automatic detection and reformulation of nonlinear equation systems as unconstrained least-squares problems with full (Gauss-Newton + second-order) Hessian computation (Section Automatic NE-to-LS Reformulation).

5. Extensive benchmarking on 727 CUTEst problems[6] and 120 Hock-Schittkowski problems,[7] with analysis of speed, accuracy, and failure modes relative to Ipopt (Section Computational Results).

6. A multi-solver fallback architecture that applies L-BFGS, augmented Lagrangian, SQP, and explicit slack reformulation when the primary interior point method fails, significantly improving robustness (Section Multi-Solver Fallback Architecture).

7. Problem preprocessing that eliminates fixed variables, detects redundant constraints, and tightens variable bounds from verified linear constraints before solving (Section Preprocessing).

8. Near-linear constraint detection that identifies linear constraints by comparing Jacobian values at multiple points, enabling the solver to skip their second-order Hessian contributions (Section Near-Linear Constraint Detection).

9. A built-in AMPL NL file interface with automatic differentiation, enabling drop-in integration with Pyomo[8] via a thin solver plugin (Section AMPL NL Interface and Pyomo Integration).

10. A C API that mirrors the Ipopt C interface, enabling direct linking from C, C++, Python (via =ctypes=/=cffi=), and other languages without subprocess overhead (Section C API).

11. A GAMS solver link that bridges GAMS's GMO (GAMS Modeling Object) API to ripopt's C API, enabling =option nlp = ripopt;= in GAMS models with automatic Hessian support, option file integration, and status mapping (Section GAMS Solver Link).

12. An iterative KKT solver based on preconditioned MINRES[9] with an incomplete LDL^T preconditioner,[10] and a hybrid direct-iterative solver that automatically switches between direct and iterative modes based on factorization cost and convergence behavior (Section Iterative Solver: Preconditioned MINRES).

2.5 Notation and Conventions

We denote the number of variables by n and the number of constraints by m. The Jacobian of g is $J \in \mathbb{R}^{m \times n}$. Following the Ipopt convention, the Lagrangian is $\mathcal{L}(x, y) = f(x) + y^T g(x)$, where $y \in \mathbb{R}^m$ are the constraint multipliers. Bound multipliers $z_L, z_U \in \mathbb{R}^n$ are associated with the variable bounds $x \geq x_L$ and $x \leq x_U$ respectively. The barrier parameter is $\mu > 0$.

3 Implicit Slack Formulation

3.1 Standard Approach: Explicit Slacks

Ipopt converts inequality constraints $g_L \leq g(x) \leq g_U$ by introducing explicit slack variables s:

$$g(x) - s = 0, \qquad g_L \leq s \leq g_U \tag{3}$$

The barrier function then includes terms $-\mu \ln(s_i - g_{L,i})$ and $-\mu \ln(g_{U,i} - s_i)$, and the KKT system has dimension $(n + n_{\text{ineq}} + m)$—primal variables, slack variables, and constraint multipliers. This requires tracking s, its associated multipliers z_s, slack fraction-to-boundary checks, and careful slack initialization.[3]

3.2 Our Approach: Implicit Slacks

We treat the constraint slacks $s_L = g(x) - g_L$ and $s_U = g_U - g(x)$ as implicit functions of x and condense their barrier contributions directly into the KKT matrix during assembly. For each inequality constraint i with slack $s_{L,i} = g_i(x) - g_{L,i} > 0$ (feasible), we compute a barrier stiffness:

$$\Sigma_{s,i} = \frac{z_{s_L,i}}{s_{L,i}} + \frac{z_{s_U,i}}{s_{U,i}} \tag{4}$$

where the implicit slack multipliers are derived from the Lagrange multiplier y_i:

$$z_{s_L,i} = \begin{cases} -y_i & \text{if } y_i < 0 \\ \mu / \tilde{s}_{L,i} & \text{otherwise} \end{cases} \tag{5}$$

and $\tilde{s}_{L,i} = \max(s_{L,i}, \mu, 10^{-10})$ is a safeguarded slack. This derivation follows from the complementarity conditions of the barrier problem: at a barrier-optimal point for a lower-bounded constraint, the multiplier y_i and the slack multiplier $z_{s_L,i}$ satisfy $y_i = -z_{s_L,i}$ (with appropriate sign conventions).

The resulting KKT system is:

$$\begin{bmatrix} H + \Sigma_x + \delta_w I & J^T \\ J & -\Sigma_s^{-1} - \delta_c I \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} r_d \\ r_p \end{bmatrix} \tag{6}$$

where $H = \nabla^2_{xx}\mathcal{L}$ is the Lagrangian Hessian, Σ_x contains the barrier contributions from variable bounds, Σ_s^{-1} is diagonal with entries $1/\Sigma_{s,i}$ for inequality constraints and 0 for equalities, and δ_w, δ_c are regularization parameters from inertia correction (Section Inertia Correction).

3.3 System Size Reduction

The implicit slack formulation reduces the KKT system size:

System	Dimension	Factorization Cost
Ipopt (explicit slacks)	$n + n_{\text{ineq}} + m$	$O((n + n_{\text{ineq}} + m)^3)$
ripopt (implicit slacks)	$n + m$	$O((n + m)^3)$
ripopt (condensed, $m \geq 2n$)	n	$O(n^2 m + n^3)$

For a problem with $n = 100$ variables and $m = 200$ inequality constraints, Ipopt factors a 500×500 system; ripopt factors a 300×300 system, or 100×100 with the Schur complement condensation.

3.4 Handling Infeasible Iterates

When an inequality constraint is violated ($s_{L,i} < 0$), no meaningful barrier term exists. We handle this with a branch: infeasible constraints receive a pure feasibility-driving right-hand side $-(g_i - g_{L,i})$ without barrier contribution, while feasible constraints use the full barrier formulation with safeguarded slack floors. This branching logic is more complex than Ipopt's uniform treatment of slacks (which are always positive by construction) but avoids the need for explicit slack variables entirely.

3.5 Condensed KKT via Schur Complement

When $m \geq 2n$, we further reduce the system by Schur complement elimination of Δy:

$$S\,\Delta x = r_d + J^T(-D_c)^{-1} r_p \tag{7}$$

where $S = H + \Sigma_x + \delta_w I + J^T(-D_c)^{-1}J$ is the Schur complement and D_c is the $(2,2)$ block diagonal. The multiplier update is recovered as $\Delta y = (-D_c)^{-1}(J\Delta x - r_p)$. For equality constraints where the $(2,2)$ entry would be zero, we use a stiff-spring penalty of -10^{-16} to avoid dropping equalities from the Schur complement. This condensation is particularly effective for problems with many more constraints than variables, such as data fitting and optimal control discretizations.

4 Convergence with Stationarity-Based Bound Multipliers

4.1 Challenges Without Explicit Slacks

In Ipopt, explicit slack variables s have well-behaved iterative multipliers z_s that naturally satisfy complementarity as the algorithm progresses. Without explicit slacks, the iterative bound multipliers z_L, z_U (updated via Newton steps with κ_σ safeguarding[3]) can exhibit two pathological behaviors:

1. **Inflation**: The κ_σ safeguard clamps $z_i \cdot s_i \in [\mu/\kappa_\sigma, \kappa_\sigma \mu]$ (with $\kappa_\sigma = 10^{10}$). When μ is still relatively high but the primal iterate is near-optimal, the safeguard can inflate z to maintain the complementarity bound, producing misleadingly large complementarity error.

2. **Oscillation**: At degenerate points—where an active variable bound coincides with an active inequality constraint—the multipliers (y, z) are not uniquely determined. The iterative z can oscillate between iterations as the Newton step alternately attributes the stationarity residual to y or z.

Using inflated or oscillating z for convergence checks leads to either stalled convergence (the complementarity error never decreases) or false convergence declarations (the dual infeasibility appears zero because z absorbs the gradient residual).

4.2 Stationarity-Based Optimal Multipliers

We compute "optimal" bound multipliers z_{opt} directly from the KKT stationarity condition. At a KKT point:

$$\nabla f(x) + J(x)^T y - z_L + z_U = 0 \tag{8}$$

For each variable i, we compute the stationarity residual $r_i = \nabla_i f + (J^T y)_i$—the component of the gradient that bound multipliers must absorb at optimality. We then set:

$$z_{L,\text{opt},i} = \begin{cases} r_i & \text{if } r_i > 0,\ x_{L,i} \text{ is finite, and complementarity gate passes} \\ 0 & \text{otherwise} \end{cases} \tag{9}$$

$$z_{U,\text{opt},i} = \begin{cases} -r_i & \text{if } r_i < 0,\ x_{U,i} \text{ is finite, and complementarity gate passes} \\ 0 & \text{otherwise} \end{cases} \tag{10}$$

The logic is: if the stationarity residual is positive and a lower bound exists, then $z_{L,i}$ should absorb it; if negative and an upper bound exists, then $z_{U,i}$ should absorb it. When the residual sign does not match any available bound, the variable contributes directly to the dual infeasibility.

4.3 The Complementarity Gate

Without a safeguard, z_{opt} can be problematic far from optimality. If the iterate is at a non-optimal point with a large gradient residual, z_{opt} would set a correspondingly large bound multiplier, making the dual infeasibility appear to be zero even though the point is not optimal. The multiplier simply absorbs the gradient residual without the point actually satisfying optimality conditions.

We introduce a complementarity gate that accepts $z_{\text{opt},i}$ only when it is consistent with the barrier problem's complementarity relationship:

$$z_{\text{opt},i} \cdot s_i \leq \kappa_{\text{compl}} \cdot \mu \tag{11}$$

with $\kappa_{\text{compl}} = 10^{10}$. When this check fails, $z_{\text{opt},i}$ is set to zero, and the full stationarity residual at variable i contributes to the dual infeasibility measure.

The gate is deliberately permissive ($\kappa_{\text{compl}} = 10^{10}$)—it only rejects z_{opt} values that are astronomically inconsistent with the current barrier parameter. As $\mu \to 0$ near convergence, the gate tightens, eventually requiring $z_{\text{opt}} \cdot s \approx 0$, which is the true complementarity condition. Without this gate, problem TP023 from the Hock-Schittkowski suite[7] falsely reports convergence at $f = 4697$ instead of the correct $f^* = 2.0$.

4.4 Two-Layer Convergence Test

We require both a scaled and an unscaled convergence check to pass:

Scaled check (adapted from Ipopt): Uses z_{opt} for dual infeasibility, with scaling factor $s_d = \max(s_{\max}, \|y\|_1 + \|z_{\text{opt}}\|_1)/s_{\max}$ where $s_{\max}$ controls the growth rate of the tolerance with multiplier magnitude.

Unscaled check: Uses iterative z for dual infeasibility against a relaxed tolerance $\epsilon_{\text{dual}} = 100$. This serves as a safety net—it catches false convergence where z_{opt} absorbs gradient residuals at non-optimal points that happen to slip through the complementarity gate.

For complementarity, we take $\min(C_z, C_{z_{\text{opt}}})$ where C_z uses iterative multipliers and $C_{z_{\text{opt}}}$ uses stationarity-based multipliers. This handles cases where iterative z is inflated by the κ_σ safeguard (making C_z misleadingly large) while z_{opt} correctly reflects the NLP complementarity.

Convergence is declared when both checks pass simultaneously.

5 Two-Phase Restoration with Multi-Attempt Recovery

5.1 Background

When the filter line search[4,3] fails to find an acceptable step at any step size $\alpha \geq \alpha_{\min}$, the solver must recover feasibility. In Ipopt, this triggers an NLP restoration phase that formulates and solves a full barrier subproblem minimizing constraint violation. If the restoration phase itself fails (e.g., the feasibility subproblem is locally infeasible), Ipopt terminates with `Restoration_Failed`. This is the dominant failure mode in practice—on the CUTEst suite, 32% of Ipopt's failures are `Restoration_Failed`.

5.2 Phase 1: Gauss-Newton Feasibility Solver

Before invoking the expensive NLP restoration, we attempt a Gauss-Newton (GN) solve on the constraint violation:[?]

$$\min_x \ \frac{1}{2}\|h(x)\|^2, \qquad h_i(x) = \max(g_{L,i} - g_i(x),\ g_i(x) - g_{U,i},\ 0) \tag{12}$$

The GN step solves the normal equations $(J_a J_a^T + \epsilon I)w = h_a$ where J_a is the Jacobian restricted to violated constraints, using Levenberg-Marquardt[11,12] regularization with adaptive ϵ (starting at 10^{-8}, increasing by $10\times$ on factorization failure). The step is $\Delta x = -J_a^T w$, a minimum-norm solution in the full variable space.

The GN phase includes three fallback mechanisms:

1. **Gradient descent fallback**: When the GN line search fails to find sufficient decrease after 30 halvings, a steepest-descent step $\Delta x = -J^T h$ is computed with a separate backtracking line search. This is critical for problems where the GN Hessian is nearly singular (e.g., when the Jacobian is rank-deficient).

2. **Proximity regularization**: When progress stalls (consecutive line search failures), a pull toward the starting point is added: $\Delta x_i \mathrel{-}= \eta(x_{\text{rest},i} - x_i)$ with $\eta = 10^{-4}(1 + k_{\text{fail}})$ increasing with consecutive failures. This prevents the GN solver from wandering far when it cannot make progress.

3. **Penalty-regularized fallback**: If the main GN loop fails to achieve sufficient violation reduction (less than 50% or less than 1%), a penalty-regularized gradient descent minimizes $\frac{1}{2}\|h\|^2 + \rho\|x - x_{\text{ref}}\|^2$ with increasing $\rho \in \{10^{-6}, 10^{-4}, 10^{-2}\}$.

The GN phase provides quadratic convergence for nonlinear equalities (vs.\ linear for gradient descent) and handles 90%+ of restoration calls at a fraction of the cost of a full NLP subproblem.

5.3 Phase 2: NLP Restoration

If GN restoration fails on two consecutive occasions, we invoke the full NLP restoration subproblem, matching Ipopt's formulation:[?]

$$\min_{x,p,n} \ \rho \sum_i (p_i + n_i) + \frac{\eta}{2}\|D_R(x - x_R)\|^2 \quad \text{s.t.} \quad g(x) - p + n = g_{\text{target}},\ p, n \geq 0 \tag{13}$$

where $\rho = 1000$, $\eta = \sqrt{\mu}$, D_R is a diagonal scaling matrix with $D_{R,i} = 1/\max(1, |x_{R,i}|)$, and x_R is the reference point (the iterate at restoration entry). This is solved by the same IPM engine with two safeguards against infinite recursion: (i) dynamic dispatch (`&dyn NlpProblem` in Rust) breaks the monomorphization chain, and (ii) `disable_nlp_restoration = tr` in the inner solve prevents further nesting.

Critically, when the NLP restoration itself returns `LocalInfeasibility` or fails to converge, we do **not** immediately terminate. Instead, the result falls through to the multi-attempt recovery mechanism described below. This design reflects our observation that the stationarity-based infeasibility detection (computed at recovery attempt 7) is more reliable than the NLP restoration's own convergence signal, which can declare local infeasibility prematurely.

5.4 Multi-Attempt Recovery

When both GN and NLP restoration fail, we cycle through up to 6 recovery attempts before declaring infeasibility:

Attempt	Strategy	Rationale
1	Mode switch (Free $\to$ Fixed, or force μ decrease)	Different barrier strategy may help
2	NLP restoration + $\mu \times 10$	Large μ smooths barrier landscape
3	$\mu \times 0.1 + x$ perturbation	Small μ may find narrow feasible path
4	$\mu \times 100 + x$ perturbation	Much larger μ for broader search
5	$\mu \times 0.01 + x$ perturbation	Much smaller μ for tight convergence
6	$\mu \times 1000 + x$ perturbation	Extreme μ increase as last resort

The μ factors $[10, 0.1, 100, 0.01, 1000, 0.001]$ alternate between increasing and decreasing by progressively larger magnitudes. This systematic exploration of different barrier landscapes is entirely novel compared to Ipopt, which terminates after a single restoration failure.

The x perturbation (attempts 3–6) uses a deterministic pattern $\Delta x_i = \text{sign}_i \cdot 10^{-4} \cdot \text{range}_i$ where $\text{sign}_i = (-1)^{[(7i+13k) \bmod 3=0]}$ depends on both the variable index i and the attempt

number k, and $\text{range}_i = x_{U,i} - x_{L,i}$ for bounded variables or $\max(|x_i|, 1)$ for unbounded. This escapes the current basin while maintaining reproducibility.

After each recovery attempt, the filter is fully reset (all entries cleared, $\theta_{\max}$ recomputed from the current point), giving the algorithm a clean slate.

After all 6 attempts, infeasibility is checked via stationarity of the constraint violation function: if $\|J^T h(x)\|_\infty < 10^{-4} \cdot \max(\theta, 1)$, where $\theta = \|h(x)\|$ is the constraint violation, we report `LocalInfeasibility`—the iterate is at a stationary point of the violation function, indicating the problem is (locally) infeasible.

5.5 Post-Restoration State Reset

After successful restoration, the algorithmic state is comprehensively reset:

- Bound multipliers: $z_i = \mu / s_i$, capped at 10^3
- Constraint multipliers: $y_i = 0$ (the IPM step will determine appropriate values via stationarity)
- Barrier parameter: $\mu = \max(\bar{c}, \mu_{\min})$ where $\bar{c}$ is the average complementarity
- Barrier mode: reset to Free (adaptive), treating restoration as a fresh start
- Filter: cleared and re-initialized from the restored point

This aggressive reset contrasts with Ipopt, which continues in whatever barrier mode was active before restoration.

6 Automatic NE-to-LS Reformulation

6.1 Problem Setting

Many CUTEst problems are nonlinear equation (NE) systems posed as constrained optimization with a dummy zero objective: find x such that $g(x) = t$ where $f(x) \equiv 0$. Ipopt treats these as standard constrained NLPs, but the zero objective provides no gradient information to guide the search direction, and Ipopt's line search—which balances objective decrease against constraint violation—cannot make switching decisions when the objective is identically flat. On the CUTEst suite, Ipopt returns status code -10 (insufficient degrees of freedom) on 20 such NE problems.

6.2 Detection

We automatically detect NE structure at the initial point by checking five conditions:

1. $m \geq n$ (at least as many equations as unknowns)
2. $|f(x_0)| < 10^{-10}$ (objective is numerically zero)

3. $\|\nabla f(x_0)\|_\infty < 10^{-10}$ (gradient is numerically zero)

4. $g_{L,i} = g_{U,i}$ for all i (all constraints are equalities)

5. $\theta(x_0) \geq 10^{-8}$ (constraints are not already satisfied; avoids unnecessary reformulation)

No user annotation is required. This detection runs once before the main solve loop and adds negligible overhead.

6.3 Reformulation as Least Squares

The detected NE problem is reformulated as an unconstrained (or bound-constrained) least-squares minimization:

$$\min_x \; F(x) = \frac{1}{2}\sum_{i=1}^{m}(g_i(x) - t_i)^2 = \frac{1}{2}\|r(x)\|^2 \tag{14}$$

where $r_i(x) = g_i(x) - t_i$ are the residuals and $t_i = g_{L,i}$ are the constraint targets. Variable bounds from the original problem are preserved. This reformulated problem is then solved by the same IPM engine as any other bound-constrained NLP.

6.4 Full Hessian with Second-Order Correction

Most least-squares solvers use only the Gauss-Newton approximation $H_{\text{GN}} = J^T J$, which is positive semi-definite and provides superlinear convergence near the solution where residuals are small.[13] However, far from the solution where residuals r_i are large, the Gauss-Newton approximation can be poor, leading to slow convergence or failure.

We compute the full Hessian of the least-squares objective:

$$\nabla^2 F(x) = J(x)^T J(x) + \sum_{i=1}^{m} r_i(x)\nabla^2 g_i(x) \tag{15}$$

The second-order correction $\sum r_i \nabla^2 g_i$ is computed by reusing the NLP Hessian evaluation interface of the original problem: calling `hessian_values(x, 0, r)` with $\sigma = 0$ (suppresses the objective Hessian, which is zero) and $\lambda = r$ (the residual vector as the multiplier). By the definition of the Lagrangian Hessian $\sigma\nabla^2 f + \sum_i \lambda_i \nabla^2 g_i$, this directly computes $\sum_i r_i \nabla^2 g_i$ without any additional interface or implementation effort.

The full Hessian is critical for problems like PFIT4, where the initial residual is $O(10^3)$ and the Gauss-Newton approximation fails to converge while the full Hessian succeeds.

6.5 Fallback and Result Interpretation

After the LS solve, we evaluate the original constraint violation $\theta = \|h(x^*)\|_\infty$ at the LS solution x^*:

- $\theta < \epsilon_{\text{tol}}$: `Optimal` (equations solved within tolerance)

- LS converged but θ large: `LocalInfeasibility` (the system is genuinely inconsistent—the LS minimum has nonzero residual)
- LS did not converge and θ large: fall back to the constrained IPM on the original problem

This fallback is important for square systems ($m = n$): the LS reformulation may converge to a local minimum of $\|r\|^2$ that is nonzero (a saddle point or local minimum in residual space), but the constrained IPM may still find a feasible point by approaching from a different direction.

7 Preprocessing

Before invoking the interior point solver, ripopt applies three preprocessing steps to simplify the problem:

7.1 Fixed Variable Elimination

When $x_{L,i} = x_{U,i}$ (within tolerance 10^{-10}), variable x_i is fixed at $(x_{L,i}+x_{U,i})/2$ and eliminated from the optimization. The `PreprocessedProblem` wrapper presents a reduced problem with remapped variable indices, delegates all function evaluations to the original problem (inserting fixed values into the full-dimensional x), and remaps the sparsity patterns of the Jacobian and Hessian. After solving the reduced problem, the solution is unmapped back to the original dimension.

7.2 Redundant Constraint Detection

Two constraints are considered redundant if they have identical bounds and identical Jacobian entries (same column structure and values) at two distinct evaluation points. Checking at two points prevents false matches from nonlinear constraints that happen to have the same Jacobian at a single point. Redundant constraints are removed from the reduced problem.

7.3 Bound Tightening from Linear Constraints

For verified linear constraints (Jacobian unchanged between two evaluation points) with exactly one free-variable nonzero entry a_j, the constraint $g_L \leq a_j x_j + c \leq g_U$ yields tighter bounds on x_j:

$$\frac{g_L - c}{a_j} \leq x_j \leq \frac{g_U - c}{a_j} \quad (\text{if } a_j > 0) \tag{16}$$

where c is the fixed-variable contribution. This tightening is only applied after verifying constraint linearity, avoiding incorrect bounds from nonlinear constraints whose Jacobian happens to have a single nonzero at the initial point.

7.4 Preprocessing Fallback

If the preprocessed solve fails (returning anything other than Optimal), ripopt automatically retries without preprocessing, since fixed variable elimination can change the numerical conditioning of borderline problems. This fallback ensures that preprocessing never causes regressions.

8 Near-Linear Constraint Detection

After scaling, ripopt evaluates the Jacobian at two distinct points (the initial point x_0 and a perturbed x_1) and compares entries per constraint row. If all entries for constraint i change by less than 10^{-12} (relative), the constraint is marked as linear. For linear constraints, $\nabla^2 g_i = 0$, so their $\lambda_i \nabla^2 g_i$ Hessian contribution is exactly zero. The solver zeros λ_i for detected linear constraints before calling the Hessian evaluation, avoiding unnecessary computation.

The perturbation $x_{1,j} = x_{0,j} + 1 + 0.1|x_{0,j}|$ is clamped to bounds, with fallback to the opposite direction if clamped back. Constraints involving variables that could not be perturbed by at least 10^{-4} are conservatively marked as nonlinear.

9 Multi-Solver Fallback Architecture

When the primary interior point method fails—returning `RestorationFailed`, `MaxIterations`, or `NumericalError`—ripopt automatically attempts up to three alternative solvers before declaring failure. This multi-solver fallback architecture is a key contributor to ripopt's robustness, recovering many problems that the primary IPM alone cannot solve.

9.1 L-BFGS for Unconstrained Problems

For unconstrained problems ($m = 0$, no finite bounds), a limited-memory BFGS (L-BFGS) solver[13] is tried **before** the IPM, since the barrier machinery is unnecessary when there are no constraints or bounds to enforce. L-BFGS maintains a compact representation of the inverse Hessian approximation using the most recent $k = 10$ curvature pairs, requiring only $O(nk)$ storage and $O(nk)$ per-iteration cost. The line search uses Wolfe conditions with backtracking.

For bound-constrained problems ($m = 0$, finite bounds), L-BFGS is used as a **fallback** after IPM failure, with a projected gradient approach that clamps iterates to the feasible box at each step.

The L-BFGS solver is particularly effective for problems where the exact Hessian is pathological far from the solution (e.g., strongly indefinite), as the quasi-Newton approximation builds up curvature information incrementally. On the CUTEst suite, L-BFGS solves several unconstrained problems (PALMER5A, PALMER7E) where the IPM's inertia correction struggles.

9.2 Augmented Lagrangian for Constrained Problems

For constrained problems that fail the primary IPM, ripopt applies a penalty-based augmented Lagrangian (AL) method using the Powell-Hestenes-Rockafellar (PHR) formulation.[13] The AL subproblem at outer iteration k is:

$$\min_x \; f(x) + \sum_i \left[y_i^{(k)} h_i(x) + \frac{\rho^{(k)}}{2} h_i(x)^2 \right] \tag{17}$$

where $h_i(x)$ is the constraint violation (accounting for inequality slacks), $y^{(k)}$ are the Lagrange multiplier estimates, and $\rho^{(k)}$ is the penalty parameter. Each AL subproblem is an unconstrained (or bound-constrained) NLP solved by the same IPM engine.

The multiplier update follows the standard rule $y_i^{(k+1)} = y_i^{(k)} + \rho^{(k)} h_i(x^*)$, and the penalty parameter increases by a factor of 10 when constraint violation fails to decrease sufficiently. The AL method converges when the constraint violation and KKT error both fall below tolerance.

The AL fallback is effective for problems where the IPM's filter line search or restoration phase fails but the feasible region is reachable via penalty methods. On CUTEst, this recovers several problems (CRESC50, TRO6X2) that resist the primary IPM.

9.3 SQP Fallback for Constrained Problems

For constrained problems where both the IPM and AL fail, ripopt tries a Sequential Quadratic Programming (SQP) solver. At each iteration, the SQP solver assembles and solves the $(n+m) \times (n+m)$ KKT system of the QP subproblem:

$$\begin{bmatrix} H & J^T \\ J & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \lambda_{\text{new}} \end{bmatrix} = \begin{bmatrix} -\nabla_x \mathcal{L} \\ -c(x) \end{bmatrix} \tag{18}$$

where H is the Hessian of the Lagrangian, J is the constraint Jacobian, and $c(x)$ is the constraint residual. Inertia correction is applied when the KKT factorization has incorrect inertia (n positive, m negative eigenvalues required). A backtracking line search with an ℓ_1 merit function $\phi(x) = f(x) + \rho \|c(x)\|_1$ determines the step size, with $\rho = \max(|\lambda_{\text{new}}|) + 1$. The SQP result replaces the current best only if it is strictly better (higher solve status or lower objective at the same status).

9.4 Explicit Slack Reformulation

For constrained problems with inequality constraints that fail both the primary IPM and the AL fallback, ripopt reformulates the problem with explicit slack variables:

$$\min_{x,s} \; f(x) \quad \text{s.t.} \quad g(x) - s = 0, \quad g_L \le s \le g_U \tag{19}$$

This converts inequality constraints into equalities with bounded slacks, producing a formulation closer to Ipopt's native representation. The reformulated problem is solved by the same IPM engine with `enable_slack_fallback = false` to prevent recursive fallback.

The explicit slack formulation addresses a fundamental limitation of the implicit slack approach: at degenerate points where constraint multipliers oscillate (Section Handling Infeasible Iterates), explicit slacks provide well-behaved iterative multiplier updates that stabilize convergence. This fallback solves problems like HATFLDH and HS83 where restoration cycling in the implicit-slack formulation prevents convergence.

9.5 Best-Point Tracking for Cycling Detection

Throughout the main IPM solve, ripopt tracks the iterate with the lowest dual infeasibility encountered during the solve. When the solver reaches the maximum iteration limit, it restores this best-dual point and checks whether it satisfies the acceptable convergence criteria. This mechanism recovers problems where the solver cycles between near-optimal points without satisfying the strict convergence test—for example, HAHN1LS, where the dual infeasibility cycles between 1.46 and 4.25×10^{-3} but the best point achieves acceptable convergence.

The primal tolerance at the max-iteration exit is relaxed to $\max(\epsilon_{\text{acc}}, \epsilon_{\text{cv,acc}})$, where $\epsilon_{\text{cv,acc}}$ is the acceptable constraint violation tolerance. This allows problems like SSINE, where constraint violation stalls at 1.4×10^{-3} after 3000 iterations, to be accepted when all other convergence criteria are met.

10 Adaptive Barrier Parameter

10.1 Simplified Oracle

Ipopt uses a quality function with golden-section search to determine the barrier parameter μ.[5] The quality function evaluates how "balanced" the iterate is at a given centering parameter $\sigma = \mu/\bar{c}$, and the golden-section search finds the σ that minimizes this function over $[\sigma_{\min}, \sigma_{\max}]$.

We replace this machinery with a direct LOQO-style formula:[?]

$$\mu = \frac{\bar{c}}{\kappa} \tag{20}$$

where $\bar{c}$ is the average complementarity (summing over both variable bound and constraint slack complementarity pairs) and $\kappa = 10$. This is equivalent to a fixed centering parameter $\sigma = 1/\kappa = 0.1$, and avoids the computational cost and implementation complexity of the golden-section search.

10.2 Free and Fixed Modes

The barrier parameter alternates between two modes:

- **Free mode**: μ is updated via Eq. (20) each iteration, and the filter is reset (all entries cleared) to give the algorithm maximum freedom with the new barrier landscape.

- **Fixed mode**: μ decreases monotonically. The barrier subproblem is considered solved when the barrier KKT error drops below $\epsilon_{\text{barrier}} \cdot \mu$, at which point μ is decreased via $\mu_{\text{new}} = \min(\kappa_\mu \cdot \mu, \mu^{\theta_\mu})$ where $\kappa_\mu = 0.2$ and $\theta_\mu = 1.5$.

The solver switches from Free to Fixed mode when insufficient progress is detected (based on a sliding window of 4 KKT error values), and back to Free mode when the Fixed mode achieves sufficient progress on a non-first iteration.

10.3 Additional Design Choices

Fraction-to-boundary rule: In Free mode, the step length bound τ uses $\tau = \max(\tau_{\min}, 1 - e_{\text{NLP}})$ where e_{NLP} is the total NLP error, rather than Ipopt's $\tau = \max(\tau_{\min}, 1 - \mu)$. Near convergence ($e_{\text{NLP}} \to 0$), $\tau \to 1$, allowing steps very close to the boundary. This is more aggressive than Ipopt's formula.

No-bounds special case: When no variable has finite bounds, μ is set to $\mu_{\min} = 10^{-11}$ immediately. With no bound constraints, the average complementarity is zero and the barrier parameter serves no purpose; setting it to $\mu_{\min}$ avoids stalling.

Post-restoration reset: After successful restoration, the barrier mode is reset to Free and the reference value history is cleared, treating restoration as a fresh start. Ipopt continues in whatever mode was active before restoration.

11 Filter Line Search

We use the filter line search of[4] as adapted by,[3] with the following implementation details.

11.1 Switching Condition

The switching condition determines whether the current iteration uses f-type (objective decrease) or h-type (constraint violation decrease) acceptance:

$$\alpha(-\nabla\phi^T d)^{s_\phi} > \delta\, \theta^{s_\theta} \quad \text{and} \quad \theta < \theta_{\min} \tag{21}$$

The inclusion of α as a multiplier on the left-hand side is critical: as α decreases during backtracking, the switching condition eventually fails, allowing the step to be accepted by h-type criteria (constraint reduction). An early version of ripopt omitted the α factor, causing the switching condition to remain active even at tiny step sizes and blocking h-type acceptance.

11.2 Second-Order Correction

We implement iterative second-order correction (SOC)[13] to address the Maratos effect, where the constraint linearization error causes rejection of good Newton steps. Up to $k_{\text{soc}} = 4$ SOC iterations are performed, each solving the same factored KKT matrix with a modified right-hand side that accounts for the nonlinear constraint residual at the trial point. A convergence check $\theta_{\text{soc}} \geq \kappa_{\text{soc}}\theta_{\text{prev}}$ with $\kappa_{\text{soc}} = 0.99$ terminates SOC early when constraint violation is not improving.

11.3 Objective Safeguard

Before checking filter acceptability, we reject any trial point where the barrier objective increases catastrophically:

$$\phi_{\text{trial}} > \phi_{\text{current}} + c_{\text{obj}}(1 + |\phi_{\text{current}}|) \tag{22}$$

with $c_{\text{obj}} = 5$. This prevents accepting steps that dramatically worsen the objective, even if they would technically pass the filter.

12 Linear Algebra

12.1 Bunch-Kaufman Factorization

For small-to-medium problems ($n + m < 100$), we use a custom dense LDL^T factorization with Bunch-Kaufman pivoting.[14] This produces the factorization $PLDL^TP^T = A$ where P is a permutation, L is unit lower triangular, and D is block diagonal with 1×1 and 2×2 blocks. The 2×2 blocks handle the indefinite structure of the KKT matrix without requiring positive definiteness.

A correctness issue in many textbook implementations of Bunch-Kaufman is that when rows and columns are swapped during pivoting, the L entries from previously computed columns must also be swapped. Without this, $PLDL^TP^T \neq A$, producing incorrect solutions that can be difficult to diagnose (the factorization appears to succeed but the solve returns wrong search directions).

12.2 Sparse LDL^T Factorization

For larger problems ($n + m \geq 100$), ripopt uses `rmumps`, a pure Rust multifrontal LDL^T solver with Bunch-Kaufman pivoting. The multifrontal method organizes the factorization into dense operations on frontal matrices defined by a supernodal elimination tree, providing better cache utilization than simplicial methods. Three fill-reducing orderings are available: AMD (approximate minimum degree, default), nested dissection[15] (Section Nested Dissection Ordering), and a KKT matching ordering that preserves primal-dual pair adjacency (Section KKT-Aware Factorization for Indefinite Systems). For constrained problems, `rmumps` is configured with KKT-aware features: threshold pivoting matching MUMPS's configuration ($u = 10^{-6}$), genuine delayed pivoting, contribution block 2×2 pivot search, matching-based scaling, and NEMIN supernode amalgamation. Ordering and symbolic analysis are computed once and cached; only the numeric factorization is repeated each iteration. The COO-to-CSC index mapping is also cached across calls, so that inertia correction refactorizations only scatter values and call the numeric phase, avoiding repeated symbolic work and memory allocation.

12.3 Inertia Correction

The KKT matrix (6) must have inertia $(n, m, 0)$—n positive and m negative eigenvalues—for the Newton step to be a descent direction for the barrier problem.[13] After factorization, we

extract the inertia from the D factor (using eigenvalues of 2×2 blocks where present) and add diagonal regularization δ_w to the $(1,1)$ block and $-\delta_c$ to the $(2,2)$ block if the inertia is incorrect.

The regularization grows exponentially: $\delta_w \to 4\delta_w$ (growth factor 4) for up to 15 attempts, with $\delta_c = 10^{-4}$ (a larger value than Ipopt's 10^{-8} to provide stronger regularization for the zero diagonal blocks of equality constraints). For unconstrained problems ($m = 0$), we first try a direct computation $\delta_w = -d_{\min} + 10^{-8}$ where $d_{\min}$ is the minimum diagonal of D, avoiding the exponential search entirely. For large systems ($n + m \geq 100$), approximate inertia within ± 1 of the expected $(n, m, 0)$ is accepted, since Bunch-Kaufman pivoting can misclassify near-zero eigenvalues at the positive/negative boundary.

A key design decision: when inertia correction fails after all 15 attempts, ripopt proceeds with the approximate factorization rather than returning an error. The filter line search will reject bad steps (producing small α), and restoration can recover. This is more pragmatic than Ipopt's approach of immediately reporting `ErrorInStepComputation`, and allows problems like EQC and HIMMELBJ to converge where Ipopt fails.

12.4 Iterative Refinement

After each KKT solve, up to 3 rounds of iterative refinement are applied using the already-factored matrix (5 rounds for large systems or when inertia correction was applied). Convergence is checked in the ℓ_∞ norm with tolerance 10^{-12}, with stagnation detection for the inertia-correction path. When inertia correction was applied, refinement targets the *original* (unperturbed) system via an analytically recovered matrix-vector product, so that the Newton direction converges to the true solution despite the regularized factorization. The backward error threshold is 10^{-6} for small systems and 10^{-4} for large systems ($n+m \geq 100$) where sparse factorization accuracy is more limited.

12.5 KKT-Aware Factorization for Indefinite Systems

Symmetric indefinite KKT systems from constrained optimization pose specific numerical challenges for sparse LDL^T factorization. Equality constraints produce zero entries on the $(2,2)$ block diagonal, creating near-zero pivots that cause catastrophic cancellation during elimination. Ipopt avoids this by relying on HSL MA57 or MUMPS (Fortran solvers with decades of refinement for indefinite systems). Since ripopt uses its own pure Rust solver (`rmumps`), we have implemented several techniques to approach production-quality factorization:

Threshold pivoting. Following MUMPS's configuration in Ipopt (=CNTL(1) = 1e-6=), `rmumps` uses a very permissive pivot threshold $u = 10^{-6}$. A pivot a_{kk} is accepted as a 1×1 pivot if $|a_{kk}| \geq u \cdot \max_{j \neq k} |a_{jk}|$. This rarely triggers delayed pivoting, preventing the cascade of delayed pivots that previously caused factorization failures on medium-scale problems.

Genuine delayed pivoting. When a column fails the threshold test, it is genuinely delayed to the parent supernode (matching MA57/MUMPS behavior) rather than force-accepted with the largest available diagonal. The parent's frontal matrix is dynamically

expanded to accommodate delayed columns from children. This is strictly better than the force-acceptance fallback and can only improve factorization quality.

Contribution block pivot search. When all fully-summed columns in a front fail the threshold test, `rmumps` searches the contribution block for a 2×2 pivot partner. Columns are classified by diagonal magnitude: a zero-diagonal column (equality constraint) is paired with a non-zero-diagonal column (primal variable), forming a 2×2 block $\begin{pmatrix} H_{jj} + \Sigma_j & J_{ij} \\ J_{ij} & 0 \end{pmatrix}$ with determinant $-J_{ij}^2$—always well-conditioned regardless of the zero diagonal. This is the key mechanism for KKT systems: it enables stable elimination of equality constraint variables that would otherwise produce catastrophic element growth. On the camshape800 benchmark (1599 variables, 1600 constraints, 800 equalities), this increases the initial step length from $\alpha = 0.027$ to $\alpha = 0.99$.

Matching-based scaling. For KKT systems (identified by the `n_primal` parameter), `rmumps` applies a two-phase scaling before factorization: (1) normalize each row/column by $1/\sqrt{\max_j |a_{ij}|}$ so the largest entry per row is $O(1)$, then (2) Ruiz iterative equilibration for fine-tuning. This approximates MUMPS's =ICNTL(8) = 77= automatic scaling and reduces the dynamic range across rows, improving pivot quality.

NEMIN supernode amalgamation. After fundamental supernode detection, consecutive small supernodes (fewer than NEMIN = 4 columns) are merged into larger blocks. This creates larger frontal matrices with more columns available for pivot selection, matching the MA57/MUMPS approach of aggressive amalgamation for better BLAS-3 utilization and pivot quality.

KKT matching ordering. An optional ordering (`KktMatchingAmd`) computes a greedy maximum-weight matching on the Jacobian, pairing each constraint with its most-coupled variable. Matched pairs are merged into super-nodes in a compressed graph, AMD is applied to the compressed graph, and super-nodes are expanded back so paired columns are adjacent in the elimination order. This enables Bunch-Kaufman to naturally form 2×2 pivots at paired locations.

12.6 Nested Dissection Ordering

For large sparse systems, fill-reducing orderings are critical for factorization efficiency. In addition to the default AMD (approximate minimum degree) ordering, `rmumps` implements a multilevel nested dissection algorithm.[15] The algorithm recursively bisects the adjacency graph of the matrix into two subgraphs connected by a small separator, then orders the separator last (so it appears at the bottom of the elimination tree). Each bisection uses a multilevel Fiduccia-Mattheyses refinement scheme[?] the graph is coarsened via heavy-edge matching, bisected at the coarsest level using breadth-first search from a pseudo-peripheral node, and then uncoarsened with boundary refinement passes that optimize cut size subject to a balance constraint ($\leq 40\%$ imbalance). Recursion terminates at subgraphs below 200 nodes, where AMD ordering is applied. For problems with regular sparsity structure (e.g., finite-element meshes), nested dissection produces $O(n \log n)$ fill versus $O(n^{3/2})$ for minimum degree.[15]

12.7 Iterative Solver: Preconditioned MINRES

For very large problems where even sparse direct factorization is too expensive, ripopt provides an iterative linear solver based on the MINRES algorithm of.[9] MINRES is a Krylov subspace method that minimizes the residual norm $\|b - Ax\|_2$ for symmetric (possibly indefinite) systems—exactly the structure of the KKT system (6). Unlike CG (which requires positive definiteness) or GMRES (which requires nonsymmetric storage), MINRES exploits the symmetric indefinite structure directly with short recurrences, requiring only $O(n)$ storage beyond the matrix.

The iterative solver uses an incomplete LDL^T factorization[10,16] as a preconditioner. The incomplete factorization reuses the same multifrontal elimination tree as the direct solver but applies dropping rules during each supernode's partial factorization: entries in L_{21} and the Schur complement (contribution block) smaller than $\tau \cdot \|L_{\cdot,j}\|$ are zeroed, where $\tau = 0.01$ is the drop tolerance. This produces a sparser approximate factorization $\tilde{L}\tilde{D}\tilde{L}^T \approx A$ that can be applied as a preconditioner in $O(\mathrm{nnz}(\tilde{L}))$ time per MINRES iteration. The pivot threshold for Bunch-Kaufman pivoting is relaxed to 0.01 (vs.\ 0.5 for the direct solver) to allow more aggressive dropping.

The inertia from the incomplete factorization is returned as an approximation of the true inertia. While not exact, it is sufficient for the IPM's inertia correction to detect gross inertia errors (e.g., zero negative eigenvalues when $m > 0$) and add appropriate regularization.

The iterative solver is selected via the =linear$_{\text{solver}}$ = Iterative= option and activates when $n + m \geq$ `sparse_threshold` (default 110). Below this threshold, the dense Bunch-Kaufman solver is used regardless of the `linear_solver` setting.

12.8 Hybrid Direct-Iterative Solver

The hybrid solver combines the reliability of direct factorization with the scalability of iterative methods. It starts in direct mode (multifrontal LDL^T) and automatically switches to iterative mode (preconditioned MINRES) under two conditions:

1. **Factorization failure**: If the direct factorization fails (e.g., numerical breakdown), the hybrid solver switches to iterative mode for subsequent iterations.

2. **Excessive factorization time**: If the direct factorization takes longer than a threshold (default 1.0 seconds wall-clock), the hybrid solver switches to iterative mode, where each "factorization" builds only the incomplete LDL^T preconditioner.

The hybrid solver also switches *back* to direct mode when the iterative solver fails: if MINRES does not converge within its iteration limit on 3 consecutive KKT solves, the solver reverts to direct factorization. This bidirectional switching prevents the solver from getting stuck in an ineffective mode.

The hybrid solver is selected via the =linear$_{\text{solver}}$ = Hybrid= option. It is particularly effective for problems where the KKT system starts well-conditioned (direct factorization is fast) but becomes increasingly ill-conditioned as the barrier parameter decreases (making iterative methods more appropriate for the later, larger-regularization iterations).

12.9 Configurable Stall Detection

The IPM includes a stall detector that terminates the solve when insufficient progress is detected over a configurable window. The `stall_iter_limit` parameter (default 30) specifies the maximum number of consecutive iterations without at least 1% improvement in either the primal infeasibility or the dual infeasibility. When the stall limit is reached, the solver checks whether the current iterate satisfies the acceptable convergence criteria before declaring `MaxIterations`. This prevents the solver from spending thousands of iterations making negligible progress on problems with slow tails, while the configurable threshold allows users to trade patience for robustness on difficult problems.

13 Implementation

13.1 Language and Dependencies

ripopt is implemented in Rust (approximately 16,000 lines of source code) with minimal external dependencies: the `rmumps` crate for sparse multifrontal LDL^T factorization and the `log` crate for diagnostic output. `rmumps` is a pure Rust implementation of a multifrontal symmetric indefinite solver, included as a workspace member in the ripopt repository (licensed separately under CeCILL-C, an LGPL-compatible free software license). `rmumps` includes AMD, nested dissection,[15] and KKT matching orderings, Bunch-Kaufman pivoting with threshold control and genuine delayed pivoting, contribution block 2×2 pivot search for KKT systems, matching-based scaling, NEMIN supernode amalgamation, and an incomplete LDL^T factorization mode[10] for preconditioning iterative solvers. The dense Bunch-Kaufman factorization, KKT assembly, filter, restoration, and all algorithmic logic are implemented from scratch. The codebase includes dedicated modules for the L-BFGS solver (`lbfgs.rs`), augmented Lagrangian method (`augmented_lagrangian.rs`), explicit slack reformulation (`slack_formulation.rs`), and iterative/hybrid linear solvers (`linear_solver/iterative.rs`, `linear_solver/hybrid.rs`). The library compiles to a single Cargo workspace with no C, C++, or Fortran dependencies, simplifying installation, cross-compilation, and embedding.

13.2 Problem Interface

The `NlpProblem` trait defines the interface for specifying optimization problems:

All evaluation methods use a buffer-filling pattern (the caller provides pre-allocated slices) to avoid heap allocation in the hot loop. The Jacobian and Hessian use sparse triplet format. The public API consists of a single function:

This interface is intentionally similar to Ipopt's `TNLP` interface, making it straightforward to port existing Ipopt problem implementations to ripopt.

13.3 Performance Characteristics

Rust's ownership model and zero-cost abstractions provide several performance advantages:

- **Stack allocation**: All per-iteration working vectors are pre-allocated before the main loop. No heap allocation occurs during iteration.
- **No interop overhead**: Unlike Ipopt, which calls into Fortran (MUMPS) or C (MA27) for each factorization, ripopt's linear algebra is in-process Rust code.
- **Cache efficiency**: The dense Bunch-Kaufman operates on contiguous memory with predictable access patterns.
- **No runtime overhead**: Rust has no garbage collector, no virtual machine, and no runtime type checking.

These advantages are most pronounced for small problems ($n + m < 50$) where per-iteration time is dominated by overhead rather than $O(n^3)$ factorization.

13.4 AMPL NL Interface and Pyomo Integration

To support integration with algebraic modeling languages, ripopt includes a built-in AMPL NL file parser and SOL file writer. The `ripopt` binary reads a problem in AMPL's NL format,[17] solves it, and writes the solution in SOL format—the same protocol used by Ipopt, KNITRO, and other AMPL-compatible solvers. The NL parser supports the full nonlinear expression DAG (directed acyclic graph) with automatic differentiation for gradient, Jacobian, and Hessian evaluation, eliminating the need for the user to supply derivative code.

This AMPL interface enables a Pyomo[8] solver plugin via Pyomo's ASL (AMPL Solver Library) infrastructure. The plugin registers `ripopt` with Pyomo's `SolverFactory`:

The plugin is a thin wrapper (30 lines of Python) that subclasses Pyomo's `ASL` solver class. Pyomo handles model serialization to NL format and solution parsing from SOL format; the plugin only needs to locate the `ripopt` executable and forward solver options. Installation is via `pip install pyomo-ripopt`, which also registers the solver through a `pyomo.solvers` entry point for automatic discovery. This means existing Pyomo workflows can switch from Ipopt to ripopt by changing a single line (`SolverFactory('ripopt')` instead of `SolverFactory('ipopt')`), with no changes to the model definition or solution retrieval code.

Solver options are passed through Pyomo's standard `solver.options` dictionary:

13.5 C API

ripopt exposes a C API that closely mirrors the Ipopt C interface, enabling direct linking from C, C++, Python (via `ctypes` or `cffi`), Julia, and any language with C FFI support—without the subprocess and file I/O overhead of the NL/SOL interface.

13.5.1 Design

The API is surfaced by compiling ripopt as a shared library (`cdylib` crate type, producing `libripopt.dylib` on macOS or `libripopt.so` on Linux). The library exports an opaque handle type and five function groups:

1. **Lifecycle**: `ripopt_create` allocates a heap-owned problem handle from bounds, sizes, and callback pointers; `ripopt_free` releases it.

2. **Options**: `ripopt_add_num_option`, `ripopt_add_int_option`, and `ripopt_add_str_option` set solver parameters by Ipopt-compatible string keys (e.g. `"tol"`, `"max_iter"`, `"mu_strategy"`).

3. **Solve**: `ripopt_solve` takes the initial point in `x` and writes the primal solution back into `x`, with optional output pointers for constraint values, the objective, and all multipliers. Any output pointer may be `NULL`.

The callback signatures are identical to the Ipopt C interface:

Each callback returns 1 on success and 0 on failure. The Jacobian and Hessian callbacks implement the standard two-call protocol: when =values == NULL=, the callback fills =iRow=/=jCol= with the sparsity pattern (0-based indices); when `values !` NULL=, it fills numerical values in the same element order. The `new_x` and `new_lambda` flags are accepted and forwarded but currently ignored (ripopt always re-evaluates), which is correct and slightly conservative.

13.5.2 Usage Example

The following skeleton shows the lifecycle for the HS071 problem (the full example is in `examples/c_api_test.c`):

Build and link from the repository root:

On HS071 the C API test produces the correct solution ($f^* \approx 17.0140$, $x^* \approx [1.000, 4.743, 3.821, 1.379]$) in 11 IPM iterations.

13.5.3 Memory Safety and Thread Safety

All raw pointer dereferences in the C API implementation are confined to `unsafe` blocks. Each `CApiProblem` handle is heap-allocated and self-contained; the solver carries no global state, so `ripopt_solve` is re-entrant and multiple handles can be solved concurrently from different threads without synchronization.

13.6 GAMS Solver Link

To support GAMS (General Algebraic Modeling System),[18] ripopt includes a standalone solver link that bridges GAMS's GMO (GAMS Modeling Object) API to ripopt's C API. The solver link is a shared library (`libGamsRipopt.dylib` on macOS, `libGamsRipopt.so` on Linux) that exports the standard GAMS solver entry points (`ripCreate`, `ripReadyAPI`, `ripCallSolver`, `ripFree`) and registers as an NLP solver in GAMS's solver configuration file.

The bridge implements five callbacks that delegate from ripopt's Ipopt-compatible callback interface to GMO evaluation functions:

- **Objective and gradient**: `gmoEvalFuncObj` and `gmoEvalGradObj` evaluate the objective function and its dense gradient. For maximization problems, both are negated (ripopt always minimizes).

- **Constraints**: `gmoEvalFunc` evaluates each constraint row individually.
- **Jacobian**: The sparsity pattern is obtained once from `gmoGetMatrixRow` in CSR format and expanded to COO. Values are computed row-by-row via `gmoEvalGrad`, extracting sparse entries from the dense gradient at the structural nonzero positions.
- **Hessian**: `gmoHessLagStruct` provides the lower-triangular sparsity in COO format. `gmoHessLagValue` evaluates the weighted Hessian of the Lagrangian, with =conweight = -1.0= to account for the GAMS multiplier sign convention ($\pi_{\text{GAMS}} = -\lambda_{\text{Ipopt}}$).

Constraint bounds are derived from GAMS equation types (===E===, ===G===, ===L===, ===N===) and the right-hand side. GAMS infinity values (=gmoPinf=/=gmoMinf=) are mapped to ripopt's $\pm 10^{30}$. When the analytical Hessian is not available (e.g., DNLP models), the solver link automatically enables L-BFGS approximation. GAMS iteration and resource limits are forwarded, and solver-specific options can be set via `ripopt.opt` files using the standard key-value format.

The solver link is built independently of the GAMS source tree (`make -C gams`) and installed by copying the library to the GAMS system directory and adding an entry to `gmscmpun.txt`. Once installed, GAMS models use ripopt with:

```
option nlp = ripopt;
Solve mymodel using nlp minimizing obj;
```

13.7 Julia/JuMP Interface

For Julia users, ripopt provides `Ripopt.jl`, a MathOptInterface (MOI)[19] wrapper that integrates with JuMP.[20] The wrapper enables ripopt to be used as a drop-in JuMP optimizer:

Solver options use the same string keys as Ipopt (`tol`, `max_iter`, `mu_strategy`, etc.) and are set via `set_optimizer_attribute`. Switching between ripopt and Ipopt requires only replacing `Ripopt.Optimizer` with `Ipopt.Optimizer`; the model definition is identical.

The implementation follows the incremental MOI interface: `Ripopt.Optimizer` implements `MOI.supports_incremental_interface`, `MOI.add_variable`, `MOI.add_constraint` (variable bounds), and `MOI.set` for `NLPBlock`, `ObjectiveSense`, `Silent`, `TimeLimitSec`, and pass-through `RawOptimizerAttribute`. Problem data (evaluator, sparsity structures, objective sign) is passed to the five C callbacks through the `user_data` pointer as a GC-protected `Ref` to a `NamedTuple`; no closures are used, ensuring compatibility with Julia on ARM (AArch64), where closure-backed `@cfunction` trampolines are not supported. The library path is resolved at load time via Julia's `__init__` hook so the precompiled image is not tied to a specific build directory.

14 Computational Results

We benchmark ripopt against Ipopt 3.14.19 (installed via Homebrew with MUMPS linear solver) on two test suites. All experiments are conducted on an Apple Mac Mini (aarch64-apple-darwin) with ripopt compiled in release mode (`-release`). Both solvers use default settings with a maximum of 3000 iterations.

14.1 Hock-Schittkowski Test Suite

The Hock-Schittkowski (HS) test suite[7] contains 120 nonlinear programming problems of varying size and difficulty, widely used for validating NLP solvers.

14.1.1 Solve Rates

Solver	Optimal	Failed	Total Solved
ripopt	118	2	118/120 (98.3%)
Ipopt (native)	116	4	116/120 (96.7%)

ripopt solves 118 of 120 problems to Optimal status, surpassing Ipopt which solves 116 of 120. Ipopt exclusively solves 0 problems that ripopt cannot. ripopt exclusively solves 2 problems (HS214, HS223) that Ipopt fails on. ripopt's 2 failures consist of 2 NumericalError.

14.1.2 Solution Quality

On the 116 problems where both solvers converge:

- Matching objectives (relative difference $< 10^{-4}$): 107/116 (92.2%)

14.1.3 Speed

Metric	ripopt	Ipopt (native)
Median per-problem time	144 μs	1.9 ms
Total time (116 problems)	35.1 ms	277.0 ms
Geometric mean speedup	12.9×	—

ripopt is faster on 114/116 commonly-solved problems (98%), with 68 problems showing 10×+ speedup. The per-iteration speedup is attributable to stack allocation, the absence of C/Fortran interop, cache-efficient dense linear algebra, and Mehrotra predictor-corrector with Gondzio centrality corrections (enabled by default).

14.1.4 Iteration Counts

Statistic	ripopt	Ipopt (native)
Mean	16.4	13.3
Median	12	10

Iteration counts are comparable. The similar iteration counts confirm that the speed advantage comes primarily from lower per-iteration cost, not fewer iterations.

14.2 CUTEst Benchmark Suite

The CUTEst suite[6] provides 727 nonlinear programming problems from diverse application domains, representing the standard benchmark for NLP solver evaluation.

14.2.1 Overall Solve Rates

Metric	ripopt	Ipopt
Metric	ripopt	Ipopt
Total solved	**569/727 (78.3%)**	556/727 (76.5%)
Both solve	524	524
ripopt only	45	—
Ipopt only	—	32

ripopt solves 13 more problems than Ipopt (569 vs. 556), reversing the previous gap. The removal of the `Acceptable` convergence status means that many problems previously counted as solved (with relaxed tolerances) are now classified as failures—primarily `NumericalError`—when they fail to achieve strict `Optimal` convergence. ripopt exclusively solves 45 problems that Ipopt cannot, while Ipopt exclusively solves 32 problems that ripopt cannot.

14.2.2 Solution Quality

On the 524 problems where both solvers converge:

- Matching objectives (relative difference $< 10^{-4}$): 416/524 (79.4%)
- 108 mismatches: both reach valid KKT points but at different local optima (including cases where fallback solvers find different basins)

14.2.3 Speed Comparison

On the 524 commonly-solved problems:

Metric	Value
Geometric mean speedup (ripopt faster)	10.2×
Median speedup (ripopt faster)	23.9×
Problems where ripopt is faster	84%
Problems where ripopt is 10×+ faster	64%

The speedup varies with problem size:

Problem size	Median speedup
Small ($n \leq 10$)	33.0×
Medium ($10 < n \leq 50$)	1.6×
Large ($n > 50$)	0.8×

The large speedup on small problems reflects Rust's lower per-iteration overhead (no Fortran interop, stack allocation, pre-allocated buffers). For medium and large problems, ripopt switches to sparse multifrontal LDL^T (via `rmumps`), which has similar asymptotic complexity to Fortran MUMPS, and the overhead advantage diminishes. Additionally, problems recovered by fallback solvers (L-BFGS, AL, SQP, slack) may require multiple solver attempts, increasing total wall-clock time on some problems.

14.2.4 Attribution of ripopt-only Solves

The 45 problems solved by ripopt but not Ipopt can be attributed to:

- **NE-to-LS reformulation** ($\sim$10 problems): Nonlinear equation systems where Ipopt returns status -10 (insufficient degrees of freedom) while ripopt detects the NE structure and reformulates as least-squares.
- **Two-phase restoration** ($\sim$8 problems): Problems where Ipopt's single NLP restoration fails but ripopt's GN phase or multi-attempt recovery succeeds.
- **Explicit slack fallback** ($\sim$8 problems): Problems where implicit-slack multiplier oscillation causes restoration cycling; the explicit slack reformulation (Section Explicit Slack Reformulation) provides well-behaved multiplier updates.
- **L-BFGS and AL fallback** ($\sim$6 problems): Unconstrained problems where L-BFGS outperforms the IPM, and constrained problems where the augmented Lagrangian succeeds after IPM failure.
- **Pragmatic inertia correction** ($\sim$5 problems): Problems where Ipopt reports `ErrorInStepComputa` but ripopt proceeds with the approximate factorization and eventually converges.
- **Other algorithmic differences** ($\sim$8 problems): Differences in the barrier parameter strategy, filter parameters, or convergence criteria that allow ripopt to converge where Ipopt does not.

Conversely, Ipopt exclusively solves 32 problems that ripopt cannot. These are primarily problems where Ipopt's barrier parameter strategy, inertia handling, or warm-start capabilities give it an advantage on specific problem structures.

14.3 Domain-Specific Benchmarks

To complement the general-purpose benchmark suites, we evaluate ripopt on three domain-specific problem sets formulated in Pyomo and solved via the AMPL NL interface.

14.3.1 Electrolyte Speciation

A set of 13 seawater electrolyte speciation problems (computing equilibrium concentrations of dissolved species) provides a challenging test of nonlinear equation solving. ripopt solves all 13 problems (100%), while Ipopt solves 12 of 13 (92.3%)—failing on the full seawater speciation problem. These problems exercise the NE-to-LS reformulation (Section Automatic NE-to-LS Reformulation) since they are square nonlinear equation systems with a dummy zero objective.

14.3.2 AC Optimal Power Flow

Four AC optimal power flow (OPF) problems test performance on structured engineering optimization. Both solvers solve all 4 problems (100%). However, ripopt is slower than Ipopt on these medium-sized constrained problems, where the sparse factorization performance of Fortran MUMPS provides an advantage.

14.3.3 Parameter Estimation

A large-scale parameter estimation problem (CHO cell growth kinetics, $n = 21{,}672$ variables, $m = 21{,}660$ constraints) tests scalability. Neither solver converges within the iteration limit (both return `MaxIterations`), indicating that this problem class requires specialized techniques beyond standard interior point methods.

14.4 Failure Analysis

14.4.1 Common Failure Patterns

From analysis of all 158 ripopt failures on CUTEst:

- **LocalInfeasibility**: 69 cases (43.7%). The dominant failure mode. The solver reaches a stationary point of the constraint violation function and declares the problem locally infeasible. Many of these problems may have feasible solutions reachable from different starting points.
- **NumericalError**: 60 cases (38.0%). The solver encounters numerical difficulties that prevent convergence to strict `Optimal` tolerance.
- **Timeout**: 15 cases (9.5%). The solver exceeds the wall-clock time limit before converging.
- **RestorationFailed**: 10 cases (6.3%). Both GN and NLP restoration fail to reduce constraint violation.
- **MaxIterations**: 4 cases (2.5%). The solver makes progress but converges too slowly within the 2999-iteration budget.

14.4.2 Comparison of Failure Modes

Failure Mode	ripopt	Ipopt
LocalInfeasibility / Infeasible	**69**	11
NumericalError	60	1
Timeout	15	10
RestorationFailed	10	4
MaxIterations	4	12
IpoptStatus(-10)	—	**123**
Other	0	10
Total failures	**158**	**171**

The failure profiles reveal fundamentally different bottlenecks. Ipopt's dominant failure mode is `IpoptStatus(-10)` (123 cases)—the CUTEst interface rejecting problems with insufficient degrees of freedom, primarily nonlinear equation systems. ripopt avoids this entirely through the NE-to-LS reformulation (Section Automatic NE-to-LS Reformulation).

ripopt's dominant failure mode is `LocalInfeasibility` (69 cases, 43.7%), followed by `NumericalError` (60 cases, 38.0%). The 158 total failures compare to Ipopt's 171, though the profiles differ qualitatively: Ipopt's dominant failure is `IpoptStatus(-10)` (123 cases) where the CUTEst interface rejects problems with insufficient degrees of freedom, a category ripopt avoids entirely through NE-to-LS reformulation. Every ripopt-reported solve achieves full convergence to the requested tolerance.

14.5 Speed vs. Accuracy Trade-off

The benchmarks reveal a clear trade-off:

- **Speed**: ripopt is dramatically faster per iteration ($\sim$100$\times$ on small problems) and moderately faster per iteration on medium problems ($\sim$10$\times$). This advantage comes from implementation characteristics (Rust, stack allocation, no interop) rather than algorithmic differences.
- **Iteration count**: ripopt uses comparable or slightly more iterations on average. The simplified μ oracle is cheaper to evaluate but occasionally less effective than Ipopt's quality-function oracle, leading to more iterations on some problems.
- **Robustness**: ripopt's two-phase restoration, multi-attempt recovery, and multi-solver fallback architecture provide complementary recovery mechanisms. ripopt exclusively solves 45 problems that Ipopt cannot, primarily through NE-to-LS reformulation, the two-phase restoration, and explicit slack fallback. With 158 failures vs. Ipopt's 171, ripopt has fewer total failures in aggregate, though the failure profiles differ: ripopt's failures are NumericalError and LocalInfeasibility, while Ipopt's are dominated by IpoptStatus(-10) rejections that ripopt avoids.
- **Solution quality**: On commonly-solved problems, solution quality is essentially identical (median relative difference 10^{-9}). The 19% mismatch rate reflects different local optima from algorithmic path differences (including fallback solvers finding different basins), not accuracy deficiencies.

15 Parametric Sensitivity Analysis

ripopt provides sIPOPT-style[21] post-optimal sensitivity analysis. Given a converged solution (x^*, y^*, z_L^*, z_U^*), the sensitivity of the optimal point to parameter perturbations Δp is computed by a single backsolve of the KKT system:

$$\begin{bmatrix} W + \Sigma & J^T \\ J & 0 \end{bmatrix} \begin{bmatrix} dx \\ dy \end{bmatrix} = - \begin{bmatrix} \nabla^2_{xp} L \cdot \Delta p \\ \frac{\partial g}{\partial p} \cdot \Delta p \end{bmatrix} \tag{23}$$

where W is the Hessian of the Lagrangian and $\Sigma = Z_L S_L^{-1} + Z_U S_U^{-1}$ is the barrier diagonal at the solution. Bound multiplier sensitivities are recovered from the complementarity conditions: $dz_{L,i} = -z_{L,i}\, dx_i/(x_i - x_{L,i})$ and $dz_{U,i} = z_{U,i}\, dx_i/(x_{U,i} - x_i)$.

15.1 Implementation

The implementation uses the simpler post-solve approach: `solve_with_sensitivity()` calls the standard `solve()`, then reassembles the *unregularized* KKT matrix at the solution point and factors it. This avoids modifying the core IPM loop entirely. The cost is one extra Hessian/Jacobian evaluation and one factorization—negligible compared to the solve itself.

The reassembled KKT omits the inertia correction terms (δ_w, δ_c) that the final IPM iteration may have applied. This is important: the regularized matrix would introduce $O(\delta)$ bias in the sensitivities, while the unregularized matrix gives the exact first-order sensitivity.

Users implement the `ParametricNlpProblem` trait, which extends `NlpProblem` with five methods specifying the number of parameters and the sparsity/values of $\partial g/\partial p$ and $\nabla^2_{xp} L$. The `SensitivityContext` retains the factored KKT and supports:

- `compute_sensitivity()`: computes dx/dp, dy/dp, dz_L/dp, dz_U/dp for one or more perturbation vectors Δp
- `reduced_hessian()`: extracts the $n \times n$ upper-left block of M^{-1} for covariance estimation in parameter estimation problems

15.2 Example: HS071 with Parametric Constraint Bound

Consider the HS071 problem where the equality constraint bound $p = 40$ is treated as a parameter. After solving at $p = 40$, sensitivity analysis predicts the solution at $p = 40.1$ by computing $x(p + \Delta p) \approx x^* + (dx/dp) \cdot \Delta p$:

Variable	$x^*(p{=}40)$	Predicted $x(p{=}40.1)$	Actual $x(p{=}40.1)$	Error
x_1	1.000000	1.000000	1.000000	$< 10^{-8}$
x_2	4.743000	4.751642	4.751634	8.4×10^{-6}
x_3	3.821150	3.824904	3.824896	8.0×10^{-6}
x_4	1.379408	1.375540	1.375553	1.4×10^{-5}

The prediction errors are $O(\Delta p^2)$ as expected from first-order Taylor expansion, confirming correctness of the sensitivity computation.

16 Opportunities for Improvement

The failure analysis and benchmark comparison reveal several concrete opportunities for improvement. The failure profile is dominated by `LocalInfeasibility` (69 cases, 43.7%), followed by `NumericalError` (60 cases, 38.0%), `Timeout` (15 cases), and `RestorationFailed` (10 cases). This profile suggests priorities for future work.

16.1 Infeasibility Detection and Recovery

The `LocalInfeasibility` failure mode (69 cases) occurs when the solver reaches a stationary point of the constraint violation function $\|\nabla_x \theta(x)\| \approx 0$. While many of these are genuinely infeasible, some may represent false declarations where a feasible solution exists in a different basin. Several strategies could reduce false positives:

- **Multi-start for NE problems**: When the LS reformulation converges to a nonzero local minimum, a second attempt from a perturbed starting point could escape the basin. This is particularly relevant for square NE systems ($m = n$) where the constrained IPM fallback currently uses the same starting point.
- **Second-order infeasibility analysis**: Using the Hessian of $\theta(x)$ at the stationary point to distinguish between local minima (genuinely infeasible) and saddle points (potentially feasible nearby) could reduce false declarations.
- **Constraint-space search**: Projecting the iterate onto the constraint manifold via a sequence of Newton steps on individual constraint subsets, rather than minimizing total violation, could find feasible points that the global violation minimization misses.

Quantifying this opportunity precisely requires distinguishing genuinely infeasible problems from false declarations, which is non-trivial. However, even a 10% reduction in `LocalInfeasibility` failures would recover approximately 7 additional problems.

16.2 Adaptive Barrier Parameter Tuning

The simplified LOQO-style oracle ($\mu = \bar{c}/\kappa$) is effective on average but occasionally produces more iterations than Ipopt's quality-function oracle. A more sophisticated oracle—potentially interpolating between the current fixed-σ approach and Ipopt's golden-section search based on detected convergence rate—could reduce iteration counts without sacrificing the simplicity that benefits the majority of problems.

16.3 Multiplier Initialization

ripopt initializes constraint multipliers y to zero and relies on the KKT system to determine appropriate values. Ipopt's least-squares multiplier initialization computes an initial y from the normal equations $J^T y \approx -\nabla f$, which can significantly reduce iteration counts, especially when the initial point is near-feasible. Adding this initialization to ripopt is straightforward and could improve convergence on problems where the initial $y = 0$ leads to poor early search directions.

16.4 Performance Optimization for Large Problems

The 16 `Timeout` failures indicate performance bottlenecks on larger problems. Two directions could help:

- **Sparse Jacobian assembly**: For over-constrained problems (e.g., EXPFITC with $m = 502$, $n = 5$), the condensed KKT reduces the system to dimension n but the dense Jacobian assembly ($O(mn)$) still dominates. Exploiting Jacobian sparsity in the condensation step could reduce per-iteration cost.
- **Sparse factorization for equality-constrained problems**: Despite significant improvements to `rmumps` (threshold pivoting, delayed pivoting, CB pivot search, matching-based scaling), the factorization quality still degrades after the first Newton step on problems with many equality constraints (e.g., camshape800), where the barrier diagonal Σ develops entries spanning many orders of magnitude. The remaining gap with MUMPS is attributable to MUMPS's MC64 maximum-weight matching, which computes a structural symmetric permutation placing large off-diagonal entries on the diagonal—a feature that is difficult to replicate without the full MC64 algorithm. Implementing a complete MC64 (or equivalent maximum-weight bipartite matching with scaling) inside `rmumps` would close this gap. The new iterative (MINRES) and hybrid solvers (Section Iterative Solver: Preconditioned MINRES) offer an alternative path for very large problems where even optimized direct factorization is prohibitive.

16.5 Quasi-Newton as Primary Solver

The L-BFGS solver (Section L-BFGS for Unconstrained Problems) is currently used as a first-attempt solver for unconstrained problems and a fallback for failed IPM solves. Extending it to serve as a primary solver for problems without second-order derivative information—or integrating L-BFGS curvature updates within the IPM framework as a Hessian approximation—could further broaden ripopt's applicability.

16.6 Scaling

ripopt implements gradient-based NLP scaling, which normalizes the objective gradient and constraint Jacobian rows to have maximum absolute value of 1.0 at the initial point. While this improves convergence on many poorly scaled problems, more sophisticated scaling strategies—such as iterative equilibration or scaling based on second-order information—could further improve performance on problems with extreme scaling imbalances.

16.7 Summary of Estimated Impact

Improvement Area	Current Failures Addressed	Estimated Problems Recovered
NumericalError recovery	112 NumericalError	20–40
Infeasibility detection	69 LocalInfeasibility	5–10
Performance optimization	16 Timeout	3–8
Adaptive μ oracle	4 MaxIterations	1–2
Multiplier initialization	Various	2–4
Scaling	Various	2–5

The largest opportunity is recovering problems that now fail as `NumericalError` after removing the `Acceptable` status. Many of these problems achieve near-convergence and

could potentially be recovered through improved convergence criteria, tighter numerical tolerances, or alternative convergence strategies. These improvements are largely independent and could collectively push ripopt's solve rate toward 80–85% of CUTEst.

17 Related Work

Interior point methods for nonlinear programming have a rich history. The barrier approach originates with,[1] and the primal-dual formulation was developed for linear programming by[22] and extended to nonlinear programming by[23] and.[3] The filter line search was introduced by[4] and adapted for interior point methods by.[3] The adaptive barrier parameter strategy used in Ipopt was formalized by.[5]

The trust region framework[24] provides an alternative globalization strategy. Condensed KKT formulations (Schur complement reductions) are well-known in the optimization literature[13] but are typically presented as optional reductions of the explicit-slack system; our use of implicit slacks as the primary formulation is less common.

Uno (Unifying Nonlinear Optimization)[25] is a recent C++ framework that decomposes Lagrange-Newton methods (SQP and interior point) into interchangeable building blocks—constraint relaxation strategies, subproblem solvers, and globalization mechanisms—that can be combined via configuration. Uno can mimic existing solvers (including Ipopt) through presets while also enabling novel combinations. In contrast, ripopt takes a fixed algorithmic approach (primal-dual barrier with filter line search) but innovates within that framework through implicit slacks, two-phase restoration, and automatic problem reformulation.

The Bunch-Kaufman factorization[14] is standard for symmetric indefinite systems. The Gauss-Newton method and Levenberg-Marquardt regularization[11,12] are standard tools for nonlinear least-squares; our contribution is their application within the restoration phase of an interior point solver and the automatic NE-to-LS reformulation with full Hessian. Parametric sensitivity analysis for NLPs was formalized in the sIPOPT framework,[21] which reuses the factored KKT matrix from the final IPM iteration to compute solution sensitivities with respect to problem parameters at near-zero cost.

The use of iterative methods for KKT systems in interior point solvers has been explored by several authors. The MINRES algorithm[9] is the natural choice for symmetric indefinite systems, as it minimizes the residual norm with short recurrences. Incomplete factorization preconditioners[10,16] are widely used to accelerate Krylov methods for sparse linear systems; our contribution is the integration of an incomplete multifrontal LDL^T factorization (sharing the same elimination tree as the direct solver) as a preconditioner within a hybrid direct-iterative framework that automatically adapts to problem difficulty. Nested dissection ordering[15] with multilevel refinement[26] is standard for fill-reducing orderings of sparse matrices from finite element discretizations.

18 Conclusions

We have presented ripopt, an interior point optimizer in approximately 16,000 lines of Rust that achieves competitive performance with Ipopt through a combination of architectural

simplification, algorithmic innovation, and a multi-solver fallback architecture. The implicit slack formulation produces smaller KKT systems and a cleaner interface. The stationarity-based convergence strategy, two-phase restoration, NE-to-LS reformulation, preprocessing, near-linear constraint detection, and fallback solvers (L-BFGS, augmented Lagrangian, SQP, explicit slack) compensate for the challenges introduced by eliminating explicit slack variables and provide novel algorithmic capabilities. A configurable linear solver infrastructure—offering direct multifrontal, iterative (preconditioned MINRES), and hybrid direct-iterative modes—enables ripopt to scale to larger problems while maintaining robustness through automatic mode switching.

On the Hock-Schittkowski test suite, ripopt solves 118 of 120 problems (98.3%) compared to Ipopt's 116 (96.7%), surpassing Ipopt. On the CUTEst benchmark suite, ripopt solves 569 of 727 problems (78.3%) compared to Ipopt's 556 (76.5%), also surpassing Ipopt. The geometric mean 10.2× speedup on commonly-solved problems—rising to 12.9× on the HS suite—makes ripopt particularly attractive for applications requiring rapid solves of small-to-medium NLPs, such as model predictive control, real-time optimization, and embedded systems. On large-scale synthetic benchmarks (up to 100K variables), ripopt solves 9 of 10 problems compared to Ipopt's 9, though Ipopt's Fortran MUMPS factorization is ∼10–15× faster per iteration on the largest systems.

The failure analysis reveals that `LocalInfeasibility` (69 cases, 43.7%) and `NumericalError` (60 cases, 38.0%) are the dominant failure modes. The `LocalInfeasibility` failures include both genuinely infeasible problems and cases where the solver incorrectly detects infeasibility. Collectively, further improvements could push the solve rate toward 85–90%.

The built-in AMPL NL interface and Pyomo solver plugin enable ripopt to serve as a drop-in replacement for Ipopt in existing Pyomo workflows, requiring only a one-line change to switch solvers. The GAMS solver link extends this accessibility to the GAMS modeling ecosystem, enabling =option nlp = ripopt;= with full Hessian support and option file integration. The sIPOPT-style parametric sensitivity analysis enables post-optimal what-if analysis and uncertainty propagation at near-zero cost, extending ripopt's utility beyond single-solve optimization.

The source code for ripopt is available at https://github.com/jkitchin/ripopt under the EPL-2.0 license.

19 References

References

[1] Anthony V Fiacco and Garth P McCormick. *Nonlinear Programming: Sequential Unconstrained Minimization Techniques*. John Wiley & Sons, 1968. URL https://doi.org/10.1137/1.9781611971316. Reprinted as SIAM Classics in Applied Mathematics 4, 1990.

[2] Anders Forsgren, Philip E Gill, and Margaret H Wright. Interior methods for nonlinear optimization. *SIAM Review*, 44(4):525–597, 2002. URL https://doi.org/10.1137/S0036144502414942.

[3] Andreas Wächter and Lorenz T Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006. URL https://doi.org/10.1007/s10107-004-0559-y.

[4] Roger Fletcher and Sven Leyffer. Nonlinear programming without a penalty function. *Mathematical Programming*, 91(2):239–269, 2002. URL https://doi.org/10.1007/s101070100244.

[5] Jorge Nocedal, Andreas Wächter, and Richard A Waltz. Adaptive barrier update strategies for nonlinear interior methods. *SIAM Journal on Optimization*, 19(4):1674–1693, 2008. URL https://doi.org/10.1137/060649513.

[6] Nicholas IM Gould, Dominique Orban, and Philippe L Toint. CUTEst: a constrained and unconstrained testing environment with safe threads for mathematical optimization. *Computational Optimization and Applications*, 60:545–557, 2015. URL https://doi.org/10.1007/s10589-014-9687-3.

[7] Willi Hock and Klaus Schittkowski. *Test Examples for Nonlinear Programming Codes*, volume 187 of *Lecture Notes in Economics and Mathematical Systems*. Springer, 1981. URL https://doi.org/10.1007/978-3-642-48320-2.

[8] Michael L Bynum, Gabriel A Hackebeil, William E Hart, Carl D Laird, Bethany L Nicholson, John D Siirola, Jean-Paul Watson, and David L Woodruff. *Pyomo—Optimization Modeling in Python*, volume 67 of *Springer Optimization and Its Applications*. Springer, 3rd edition, 2021. URL https://doi.org/10.1007/978-3-030-68928-5.

[9] Christopher C Paige and Michael A Saunders. Solution of sparse indefinite systems of linear equations. *SIAM Journal on Numerical Analysis*, 12(4):617–629, 1975. URL https://doi.org/10.1137/0712047.

[10] J A Meijerink and H A van der Vorst. An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-matrix. *Mathematics of Computation*, 31 (137):148–162, 1977. URL https://doi.org/10.1090/S0025-5718-1977-0438681-4.

[11] Kenneth Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2:164–168, 1944. URL https://doi.org/10.1090/qam/10666.

[12] Donald W Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963. URL https://doi.org/10.1137/0111030.

[13] Jorge Nocedal and Stephen J Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, 2nd edition, 2006. URL https://doi.org/10.1007/978-0-387-40065-5.

[14] James R Bunch and Linda Kaufman. Some stable methods for calculating inertia and solving symmetric linear systems. *Mathematics of Computation*, 31(137):163–179, 1977. URL https://doi.org/10.1090/S0025-5718-1977-0428694-0.

[15] J Alan George. Nested dissection of a regular finite element mesh. *SIAM Journal on Numerical Analysis*, 10(2):345–363, 1973. URL https://doi.org/10.1137/0710032.

[16] Edmond Chow and Yousef Saad. Experimental study of ILU preconditioners for indefinite matrices. *Journal of Computational and Applied Mathematics*, 86(2):387–414, 1997. URL https://doi.org/10.1016/S0377-0427(97)00171-2.

[17] David M Gay. Writing .nl files. Technical Report SAND2003-3492, Sandia National Laboratories, 2003. Updated 2005. Available at https://ampl.com/resources/hooking-your-solver-to-ampl/.

[18] Michael R Bussieck and Alex Meeraus. The GAMS system. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 137–157. Springer, 2004. URL https://doi.org/10.1007/978-1-4613-0215-5_8.

[19] Benoît Legat, Oscar Dowson, Joaquim Dias Garcia, and Miles Lubin. MathOptInterface: a data structure for mathematical optimization problems. *INFORMS Journal on Computing*, 34(2):672–689, 2022. URL https://doi.org/10.1287/ijoc.2021.1083.

[20] Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, and Juan Pablo Vielma. JuMP 1.0: recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, 15:581–589, 2023. URL https://doi.org/10.1007/s12532-023-00239-3.

[21] Hans Pirnay, Rodrigo López-Negrete, and Lorenz T Biegler. Optimal sensitivity based on IPOPT. *Mathematical Programming Computation*, 4(4):307–331, 2012. URL https://doi.org/10.1007/s12532-012-0043-2.

[22] Sanjay Mehrotra. On the implementation of a primal-dual interior point method. *SIAM Journal on Optimization*, 2(4):575–601, 1992. URL https://doi.org/10.1137/0802028.

[23] Richard H Byrd, Mary E Hribar, and Jorge Nocedal. An interior point algorithm for large-scale nonlinear programming. *SIAM Journal on Optimization*, 9(4):877–900, 1999. URL https://doi.org/10.1137/S1052623497325107.

[24] Andrew R Conn, Nicholas IM Gould, and Philippe L Toint. *Trust Region Methods*. MPS/SIAM Series on Optimization. SIAM, 2000. URL https://doi.org/10.1137/1.9780898719857.

[25] Charlie Vanaret and Sven Leyffer. Implementing a unified solver for nonlinearly constrained optimization. Submitted to Mathematical Programming Computation. Preprint available at https://www.researchgate.net/publication/397446552, 2024.

[26] George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998. URL https://doi.org/10.1137/S1064827595287997.